

Tree-Based Methods

Regression Trees, Classification Trees, and
Pruning

Dr. Rajesh Kumar Mundotiya

Assistant Professor, Department of Computer Science
and Engineering

Lab In-Charge: MATRA Lab, Indian Institute of
Technology Bhilai



MATRA LAB

Session Roadmap

Recap & Objectives

Mathematical Prerequisites

Regression Trees

Classification Trees

Trees vs Linear Models

Applied View

Summary



SECTION 1

Recap & Objectives

Where We Were: Moving Beyond Linearity

From the previous session

Splines and GAMs extended the linear model while keeping it *smooth*: each predictor's effect is a smooth global function, built from a basis expansion (truncated power basis, natural splines) and fit by (penalized) least squares.

The gap this session fills

Trees take a completely different approach. Instead of a smooth global function of x , a **tree** *recursively partitions* the predictor space into simple rectangular regions and predicts a *constant* (regression) or a *class* (classification) within each region — no smoothness, no global equation, just a sequence of IF-THEN splits.

Where We Were: Moving Beyond Linearity

From the previous session

Splines and GAMs extended the linear model while keeping it *smooth*: each predictor's effect is a smooth global function, built from a basis expansion (truncated power basis, natural splines) and fit by (penalized) least squares.

The gap this session fills

Trees take a completely different approach. Instead of a smooth global function of x , a **tree** *recursively partitions* the predictor space into simple rectangular regions and predicts a *constant* (regression) or a *class* (classification) within each region — no smoothness, no global equation, just a sequence of IF-THEN splits.

Learning Objectives

By the end of this session you will be able to:

1. Explain how recursive binary splitting partitions the feature space and produces regionwise constant predictions.
2. Derive the RSS-minimizing split for a regression tree.
3. Explain overfitting in trees and derive cost-complexity (weakest-link) pruning, including how α is chosen by CV.
4. Compute and compare the three classification-tree impurity measures: error rate, Gini index, entropy.
5. Contrast trees with linear models and list their key advantages and disadvantages — setting up why we need *ensembles* of trees.

Learning Objectives

By the end of this session you will be able to:

1. Explain how recursive binary splitting partitions the feature space and produces regionwise constant predictions.
2. Derive the RSS-minimizing split for a regression tree.
3. Explain overfitting in trees and derive cost-complexity (weakest-link) pruning, including how α is chosen by CV.
4. Compute and compare the three classification-tree impurity measures: error rate, Gini index, entropy.
5. Contrast trees with linear models and list their key advantages and disadvantages — setting up why we need *ensembles* of trees.

Learning Objectives

By the end of this session you will be able to:

1. Explain how recursive binary splitting partitions the feature space and produces regionwise constant predictions.
2. Derive the RSS-minimizing split for a regression tree.
3. Explain overfitting in trees and derive cost-complexity (weakest-link) pruning, including how α is chosen by CV.
4. Compute and compare the three classification-tree impurity measures: error rate, Gini index, entropy.
5. Contrast trees with linear models and list their key advantages and disadvantages — setting up why we need *ensembles* of trees.

Learning Objectives

By the end of this session you will be able to:

1. Explain how recursive binary splitting partitions the feature space and produces regionwise constant predictions.
2. Derive the RSS-minimizing split for a regression tree.
3. Explain overfitting in trees and derive cost-complexity (weakest-link) pruning, including how α is chosen by CV.
4. Compute and compare the three classification-tree impurity measures: error rate, Gini index, entropy.
5. Contrast trees with linear models and list their key advantages and disadvantages — setting up why we need *ensembles* of trees.

Learning Objectives

By the end of this session you will be able to:

1. Explain how recursive binary splitting partitions the feature space and produces regionwise constant predictions.
2. Derive the RSS-minimizing split for a regression tree.
3. Explain overfitting in trees and derive cost-complexity (weakest-link) pruning, including how α is chosen by CV.
4. Compute and compare the three classification-tree impurity measures: error rate, Gini index, entropy.
5. Contrast trees with linear models and list their key advantages and disadvantages — setting up why we need *ensembles* of trees.

Learning Objectives

By the end of this session you will be able to:

1. Explain how recursive binary splitting partitions the feature space and produces regionwise constant predictions.
2. Derive the RSS-minimizing split for a regression tree.
3. Explain overfitting in trees and derive cost-complexity (weakest-link) pruning, including how α is chosen by CV.
4. Compute and compare the three classification-tree impurity measures: error rate, Gini index, entropy.
5. Contrast trees with linear models and list their key advantages and disadvantages — setting up why we need *ensembles* of trees.

1. Recursive binary splitting

A single feature j and threshold s split a region R into two half-spaces

$$R_1(j, s) = \{\mathbf{x} \mid x_j < s\}, \quad R_2(j, s) = \{\mathbf{x} \mid x_j \geq s\}.$$

Applying this recursively to each child region builds the tree.

2. Minimizing an error/impurity criterion at each split

At every node we pick (j, s) to minimize a criterion evaluated on the two children: RSS for a continuous response, or an *impurity measure* (error rate, Gini, entropy) for a categorical response. The tree never looks more than one split ahead — it is a **greedy** algorithm.

1. Recursive binary splitting

A single feature j and threshold s split a region R into two half-spaces

$$R_1(j, s) = \{\mathbf{x} \mid x_j < s\}, \quad R_2(j, s) = \{\mathbf{x} \mid x_j \geq s\}.$$

Applying this recursively to each child region builds the tree.

2. Minimizing an error/impurity criterion at each split

At every node we pick (j, s) to minimize a criterion evaluated on the two children: RSS for a continuous response, or an *impurity measure* (error rate, Gini, entropy) for a categorical response. The tree never looks more than one split ahead — it is a **greedy** algorithm.

From a Global Equation to a Partition

Key idea

A **regression tree** does not learn one global equation for $Y = f(\mathbf{x})$. It divides the predictor space into M disjoint regions $R_1, \dots, R_M$ and predicts a *constant* in each:

$$\hat{f}(\mathbf{x}) = \sum_{m=1}^M c_m \mathbb{I}(\mathbf{x} \in R_m), \quad \hat{c}_m = \text{average of } y_i \text{ for } \mathbf{x}_i \in R_m.$$

Why rectangles?

Recursive binary splitting on one feature at a time always produces axis-aligned rectangular (box-shaped) regions — easy to describe, easy to visualize as a tree of yes/no questions.

From a Global Equation to a Partition

Key idea

A **regression tree** does not learn one global equation for $Y = f(\mathbf{x})$. It divides the predictor space into M disjoint regions $R_1, \dots, R_M$ and predicts a *constant* in each:

$$\hat{f}(\mathbf{x}) = \sum_{m=1}^M c_m \mathbb{I}(\mathbf{x} \in R_m), \quad \hat{c}_m = \text{average of } y_i \text{ for } \mathbf{x}_i \in R_m.$$

Why rectangles?

Recursive binary splitting on one feature at a time always produces axis-aligned rectangular (box-shaped) regions — easy to describe, easy to visualize as a tree of yes/no questions.

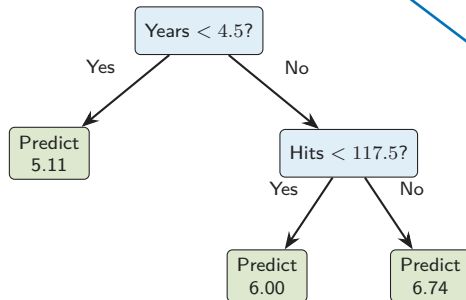
Worked Example: Predicting Baseball Salary

Setup

Predict $\log(\text{Salary})$ from **Years** (experience) and **Hits**, plus other stats (Home Runs, Runs, RBI).

Regions found

- ▶ R_1 : Years $< 4.5 \Rightarrow$ predict 5.11
- ▶ R_2 : Years ≥ 4.5 , Hits $< 117.5 \Rightarrow 6.00$
- ▶ R_3 : Years ≥ 4.5 , Hits $\geq 117.5 \Rightarrow 6.74$



Two Equivalent Pictures: Tree and Partition

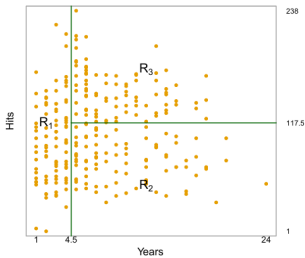


FIGURE 8.2. The three-region partition for the Hitters data set from the regression tree illustrated in Figure 8.1.

Reading the picture

The tree of yes/no questions (left, previous slide) and the partitioned (Years, Hits) plane (right) show *the same model*: each terminal node $\equiv$ one rectangular region R_m with its own constant prediction $\hat{c}_m$.

Prediction rule

$$\hat{y} = \hat{c}_m \quad \text{if } \mathbf{x} \text{ falls in region } R_m.$$

Finding the Best Split: Minimizing RSS

Objective at each node

$$\min_{j,s} \left[\sum_{\mathbf{x}_i \in R_1(j,s)} (y_i - \hat{c}_1)^2 + \sum_{\mathbf{x}_i \in R_2(j,s)} (y_i - \hat{c}_2)^2 \right]$$

- ▶ j : the feature to split on; s : the threshold.
- ▶ $R_1(j, s), R_2(j, s)$: the resulting left/right regions.
- ▶ $\hat{c}_1, \hat{c}_2$: the mean response within each region — the optimal constant for squared error is always the mean.

In practice

For every feature, scan every candidate threshold, compute the combined RSS, and keep the (j, s) with the smallest RSS. Recurse on each child.

Finding the Best Split: Minimizing RSS

Objective at each node

$$\min_{j,s} \left[\sum_{\mathbf{x}_i \in R_1(j,s)} (y_i - \hat{c}_1)^2 + \sum_{\mathbf{x}_i \in R_2(j,s)} (y_i - \hat{c}_2)^2 \right]$$

- ▶ j : the feature to split on; s : the threshold.
- ▶ $R_1(j, s), R_2(j, s)$: the resulting left/right regions.
- ▶ $\hat{c}_1, \hat{c}_2$: the mean response within each region — the optimal constant for squared error is always the mean.

In practice

For every feature, scan every candidate threshold, compute the combined RSS, and keep the (j, s) with the smallest RSS. Recurse on each child.

Finding the Best Split: Minimizing RSS

Objective at each node

$$\min_{j,s} \left[\sum_{\mathbf{x}_i \in R_1(j,s)} (y_i - \hat{c}_1)^2 + \sum_{\mathbf{x}_i \in R_2(j,s)} (y_i - \hat{c}_2)^2 \right]$$

- ▶ j : the feature to split on; s : the threshold.
- ▶ $R_1(j, s), R_2(j, s)$: the resulting left/right regions.
- ▶ $\hat{c}_1, \hat{c}_2$: the mean response within each region — the optimal constant for squared error is always the mean.

In practice

For every feature, scan every candidate threshold, compute the combined RSS, and keep the (j, s) with the smallest RSS. Recurse on each child.

Recursive Binary Splitting: The Algorithm

1. Start with the full training set as one region.
2. For every feature, evaluate every possible split point.
3. Compute the RSS for each candidate split.
4. Choose the split that minimizes total RSS; divide into two child regions.
5. Repeat independently within each child.
6. Stop when a stopping criterion is reached (e.g. minimum node size).

Why “recursive” and “binary”

Each split creates exactly *two* children, and the same procedure is applied again inside each child — hence **recursive binary splitting**.

Recursive Binary Splitting: The Algorithm

1. Start with the full training set as one region.
2. For every feature, evaluate every possible split point.
3. Compute the RSS for each candidate split.
4. Choose the split that minimizes total RSS; divide into two child regions.
5. Repeat independently within each child.
6. Stop when a stopping criterion is reached (e.g. minimum node size).

Why “recursive” and “binary”

Each split creates exactly *two* children, and the same procedure is applied again inside each child — hence **recursive binary splitting**.

Recursive Binary Splitting: The Algorithm

1. Start with the full training set as one region.
2. For every feature, evaluate every possible split point.
3. Compute the RSS for each candidate split.
4. Choose the split that minimizes total RSS; divide into two child regions.
5. Repeat independently within each child.
6. Stop when a stopping criterion is reached (e.g. minimum node size).

Why “recursive” and “binary”

Each split creates exactly *two* children, and the same procedure is applied again inside each child — hence **recursive binary splitting**.

Recursive Binary Splitting: The Algorithm

1. Start with the full training set as one region.
2. For every feature, evaluate every possible split point.
3. Compute the RSS for each candidate split.
4. Choose the split that minimizes total RSS; divide into two child regions.
5. Repeat independently within each child.
6. Stop when a stopping criterion is reached (e.g. minimum node size).

Why “recursive” and “binary”

Each split creates exactly *two* children, and the same procedure is applied again inside each child — hence **recursive binary splitting**.

Recursive Binary Splitting: The Algorithm

1. Start with the full training set as one region.
2. For every feature, evaluate every possible split point.
3. Compute the RSS for each candidate split.
4. Choose the split that minimizes total RSS; divide into two child regions.
5. Repeat independently within each child.
6. Stop when a stopping criterion is reached (e.g. minimum node size).

Why “recursive” and “binary”

Each split creates exactly *two* children, and the same procedure is applied again inside each child — hence **recursive binary splitting**.

Recursive Binary Splitting: The Algorithm

1. Start with the full training set as one region.
2. For every feature, evaluate every possible split point.
3. Compute the RSS for each candidate split.
4. Choose the split that minimizes total RSS; divide into two child regions.
5. Repeat independently within each child.
6. Stop when a stopping criterion is reached (e.g. minimum node size).

Why “recursive” and “binary”

Each split creates exactly *two* children, and the same procedure is applied again inside each child — hence **recursive binary splitting**.

Recursive Binary Splitting: The Algorithm

1. Start with the full training set as one region.
2. For every feature, evaluate every possible split point.
3. Compute the RSS for each candidate split.
4. Choose the split that minimizes total RSS; divide into two child regions.
5. Repeat independently within each child.
6. Stop when a stopping criterion is reached (e.g. minimum node size).

Why “recursive” and “binary”

Each split creates exactly *two* children, and the same procedure is applied again inside each child — hence **recursive binary splitting**.

A Greedy, Not Globally Optimal, Algorithm

Greedy strategy

At each node the algorithm picks the split with the largest *immediate* reduction in RSS — it does *not* search over all possible trees for the globally best one (that search is combinatorially infeasible).

Candidate split	Total RSS
Years < 4.5	40
Runs < 60	47
Hits < 120	52
RBI < 75	61

Consequence

The algorithm immediately commits to **Years < 4.5** (smallest RSS) and never revisits that choice — a split that looks locally best may not be part of the globally best tree.

A Greedy, Not Globally Optimal, Algorithm

Greedy strategy

At each node the algorithm picks the split with the largest *immediate* reduction in RSS — it does *not* search over all possible trees for the globally best one (that search is combinatorially infeasible).

Candidate split	Total RSS
Years < 4.5	40
Runs < 60	47
Hits < 120	52
RBI < 75	61

Consequence

The algorithm immediately commits to **Years < 4.5** (smallest RSS) and never revisits that choice — a split that looks locally best may not be part of the globally best tree.

A Greedy, Not Globally Optimal, Algorithm

Greedy strategy

At each node the algorithm picks the split with the largest *immediate* reduction in RSS — it does *not* search over all possible trees for the globally best one (that search is combinatorially infeasible).

Candidate split	Total RSS
Years < 4.5	40
Runs < 60	47
Hits < 120	52
RBI < 75	61

Consequence

The algorithm immediately commits to **Years** < 4.5 (smallest RSS) and never revisits that choice — a split that looks locally best may not be part of the globally best tree.

When Should We Stop Splitting?

Keep splitting: pros

- ▶ Training RSS keeps decreasing.
- ▶ Leaves become more homogeneous.
- ▶ Better *training* fit.

Keep splitting: cons

- ▶ Tree becomes very complex.
- ▶ Memorizes the training data.
- ▶ Poor prediction on unseen data.

Observation

A tree with one observation per leaf has near-zero training error but usually performs poorly on new data — classic **overfitting**.

When Should We Stop Splitting?

Keep splitting: pros

- ▶ Training RSS keeps decreasing.
- ▶ Leaves become more homogeneous.
- ▶ Better *training* fit.

Keep splitting: cons

- ▶ Tree becomes very complex.
- ▶ Memorizes the training data.
- ▶ Poor prediction on unseen data.

Observation

A tree with one observation per leaf has near-zero training error but usually performs poorly on new data — classic **overfitting**.

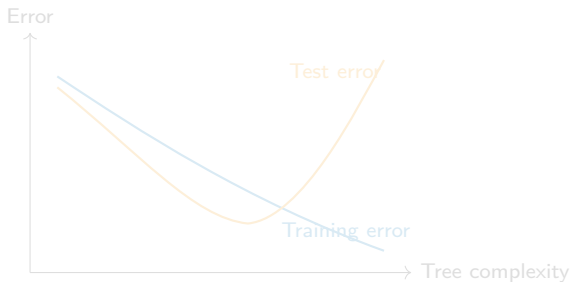
Overfitting: Bias–Variance Tradeoff in Trees

Small tree

- ▶ Higher bias.
- ▶ Lower variance.
- ▶ Better generalization.

Very large tree

- ▶ Lower bias.
- ▶ Higher variance.
- ▶ Overfits training data.



Key observation

Training error always falls as the tree grows; test error falls, then *rises* once the tree

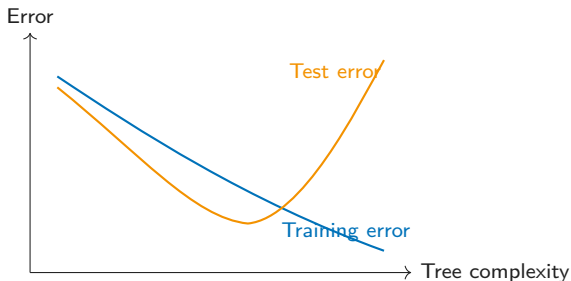
Overfitting: Bias–Variance Tradeoff in Trees

Small tree

- ▶ Higher bias.
- ▶ Lower variance.
- ▶ Better generalization.

Very large tree

- ▶ Lower bias.
- ▶ Higher variance.
- ▶ Overfits training data.



Key observation

Dr. Rajesh Kumar Mundotiya Training error always falls as the tree grows; test error falls, then *rises* once the tree

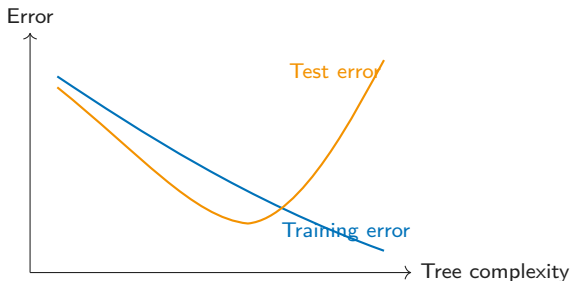
Overfitting: Bias–Variance Tradeoff in Trees

Small tree

- ▶ Higher bias.
- ▶ Lower variance.
- ▶ Better generalization.

Very large tree

- ▶ Lower bias.
- ▶ Higher variance.
- ▶ Overfits training data.



Key observation

Training error always falls as the tree grows; test error falls, then *rises* once the tree

Why Not Just Stop Early?

The problem with greedy early stopping

Requiring a minimum RSS decrease per split is too short-sighted: a seemingly weak split can be followed by a very good one further down the tree. Stopping early can miss that good split entirely.

Better strategy

Grow a large tree T_0 first, then **prune** it back — this lets us evaluate whole subtrees rather than committing split-by-split.

Why Not Just Stop Early?

The problem with greedy early stopping

Requiring a minimum RSS decrease per split is too short-sighted: a seemingly weak split can be followed by a very good one further down the tree. Stopping early can miss that good split entirely.

Better strategy

Grow a large tree T_0 first, then **prune** it back — this lets us evaluate whole subtrees rather than committing split-by-split.

Cost-Complexity (Weakest-Link) Pruning

Idea

Find the subtree with the best balance of training accuracy and complexity. A large tree has low training error but high complexity; a smaller tree has slightly higher training error but generalizes better.

Large tree $\implies$ Prune $\implies$ Smaller, generalizable tree

Why not try every subtree?

A tree with k internal nodes has an astronomical number of possible subtrees. Cost-complexity pruning indexes candidate subtrees by a single tunable parameter α instead of searching all of them.

Cost-Complexity (Weakest-Link) Pruning

Idea

Find the subtree with the best balance of training accuracy and complexity. A large tree has low training error but high complexity; a smaller tree has slightly higher training error but generalizes better.

Large tree $\implies$ Prune $\implies$ Smaller, generalizable tree

Why not try every subtree?

A tree with k internal nodes has an astronomical number of possible subtrees. Cost-complexity pruning indexes candidate subtrees by a single tunable parameter α instead of searching all of them.

Cost-Complexity (Weakest-Link) Pruning

Idea

Find the subtree with the best balance of training accuracy and complexity. A large tree has low training error but high complexity; a smaller tree has slightly higher training error but generalizes better.

Large tree $\implies$ Prune $\implies$ Smaller, generalizable tree

Why not try every subtree?

A tree with k internal nodes has an astronomical number of possible subtrees. Cost-complexity pruning indexes candidate subtrees by a single tunable parameter α instead of searching all of them.

The Cost-Complexity Objective

$$C_{\alpha}(T) = \text{RSS}(T) + \alpha |T|$$

Where

- ▶ $\text{RSS}(T)$: training error of subtree T .
- ▶ $|T|$: number of terminal (leaf) nodes.
- ▶ $\alpha \geq 0$: complexity penalty per leaf.

Interpretation

- ▶ Small $\alpha \Rightarrow$ larger tree.
- ▶ Large $\alpha \Rightarrow$ smaller tree.
- ▶ $\alpha = 0$: unpruned tree T_0 .

Observation

For each α there is a unique smallest subtree of T_0 minimizing $C_{\alpha}(T)$ — as α increases, weakest branches (smallest per-leaf RSS improvement) get pruned first, giving a nested sequence of subtrees.

The Cost-Complexity Objective

$$C_{\alpha}(T) = \text{RSS}(T) + \alpha |T|$$

Where

- ▶ $\text{RSS}(T)$: training error of subtree T .
- ▶ $|T|$: number of terminal (leaf) nodes.
- ▶ $\alpha \geq 0$: complexity penalty per leaf.

Interpretation

- ▶ Small $\alpha \Rightarrow$ larger tree.
- ▶ Large $\alpha \Rightarrow$ smaller tree.
- ▶ $\alpha = 0$: unpruned tree T_0 .

Observation

For each α there is a unique smallest subtree of T_0 minimizing $C_{\alpha}(T)$ — as α increases, weakest branches (smallest per-leaf RSS improvement) get pruned first, giving a nested sequence of subtrees.

The Cost-Complexity Objective

$$C_{\alpha}(T) = \text{RSS}(T) + \alpha |T|$$

Where

- ▶ $\text{RSS}(T)$: training error of subtree T .
- ▶ $|T|$: number of terminal (leaf) nodes.
- ▶ $\alpha \geq 0$: complexity penalty per leaf.

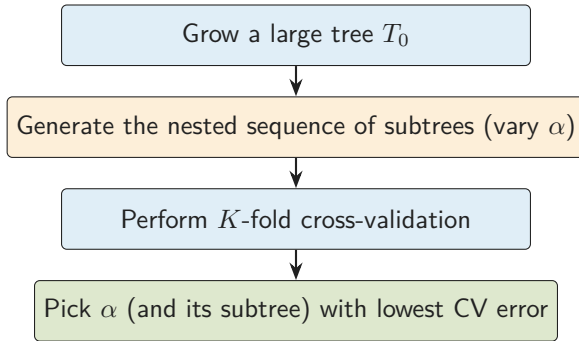
Interpretation

- ▶ Small $\alpha \Rightarrow$ larger tree.
- ▶ Large $\alpha \Rightarrow$ smaller tree.
- ▶ $\alpha = 0$: unpruned tree T_0 .

Observation

For each α there is a unique smallest subtree of T_0 minimizing $C_{\alpha}(T)$ — as α increases, weakest branches (smallest per-leaf RSS improvement) get pruned first, giving a nested sequence of subtrees.

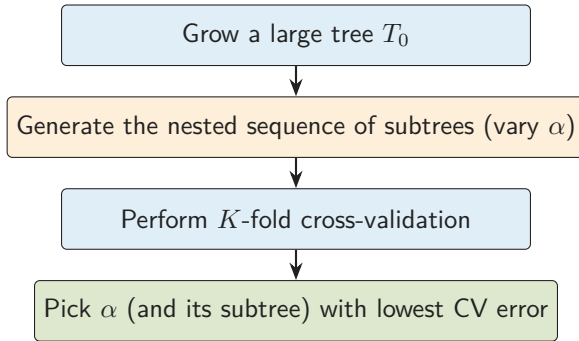
Selecting α by Cross-Validation



Result

Cost-complexity pruning with CV-selected α turns tree-growing from a purely greedy, hard-to-tune procedure into a single well-defined model-selection problem in one hyperparameter.

Selecting α by Cross-Validation



Result

Cost-complexity pruning with CV-selected α turns tree-growing from a purely greedy, hard-to-tune procedure into a single well-defined model-selection problem in one hyperparameter.

Regression Trees vs Classification Trees

Regression tree

- ▶ Target variable is continuous.
- ▶ Predicts the *mean* response in a leaf.
- ▶ Split criterion: minimize RSS.
- ▶ Examples: salary, house price prediction.

Classification tree

- ▶ Target variable is categorical.
- ▶ Predicts the *majority class* in a leaf.
- ▶ Split criterion: maximize node purity.
- ▶ Examples: spam detection, disease diagnosis.

Key difference

Regression trees predict numbers; classification trees predict class labels — but both are built by the *same* recursive binary splitting recipe, only the splitting criterion changes.

Regression Trees vs Classification Trees

Regression tree

- ▶ Target variable is continuous.
- ▶ Predicts the *mean* response in a leaf.
- ▶ Split criterion: minimize RSS.
- ▶ Examples: salary, house price prediction.

Classification tree

- ▶ Target variable is categorical.
- ▶ Predicts the *majority class* in a leaf.
- ▶ Split criterion: maximize node purity.
- ▶ Examples: spam detection, disease diagnosis.

Key difference

Regression trees predict numbers; classification trees predict class labels — but both are built by the *same* recursive binary splitting recipe, only the splitting criterion changes.

What Is Node Purity?

A node is **pure** if all its observations belong to the same class.

Pure node

Spam
Spam
Spam
Spam
Spam

Purity = 100%

Impure node

Spam
Not Spam
Spam
Not Spam
Spam

Mixed classes

Goal

Every split should increase the purity of its child nodes — this is the classification analogue of “decrease RSS.”

What Is Node Purity?

A node is **pure** if all its observations belong to the same class.

Pure node

Spam
Spam
Spam
Spam
Spam

Purity = 100%

Impure node

Spam
Not Spam
Spam
Not Spam
Spam

Mixed classes

Goal

Every split should increase the purity of its child nodes — this is the classification analogue of “decrease RSS.”

Formula

$$E = 1 - \max_k(\hat{p}_{mk})$$

Notation

$\hat{p}_{mk}$: proportion of class- k observations in node m .

Example

Node has Spam= 8, Not-Spam= 2:

$$\hat{p} = \frac{8}{10} = 0.8, \quad E = 1 - 0.8 = 0.2.$$

Limitation

E is not sensitive enough for tree-growing: it can stay flat across splits that clearly improve purity, because it only looks at the single largest class proportion.

Formula

$$E = 1 - \max_k(\hat{p}_{mk})$$

Notation

$\hat{p}_{mk}$: proportion of class- k observations in node m .

Example

Node has Spam= 8, Not-Spam= 2:

$$\hat{p} = \frac{8}{10} = 0.8, \quad E = 1 - 0.8 = 0.2.$$

Limitation

E is not sensitive enough for tree-growing: it can stay flat across splits that clearly improve purity, because it only looks at the single largest class proportion.

Classification Error Rate

Formula

$$E = 1 - \max_k(\hat{p}_{mk})$$

Notation

$\hat{p}_{mk}$: proportion of class- k observations in node m .

Example

Node has Spam= 8, Not-Spam= 2:

$$\hat{p} = \frac{8}{10} = 0.8, \quad E = 1 - 0.8 = 0.2.$$

Limitation

E is not sensitive enough for tree-growing: it can stay flat across splits that clearly improve purity, because it only looks at the single largest class proportion.

The Gini Index

Formula

$$G = \sum_{k=1}^K \hat{p}_{mk}(1 - \hat{p}_{mk}) = 1 - \sum_{k=1}^K \hat{p}_{mk}^2$$

Node: 8 Spam, 2 Not Spam

$$\hat{p}_1 = 0.8, \hat{p}_2 = 0.2$$

Calculation

$$G = 1 - (0.8^2 + 0.2^2) = 1 - 0.68 = 0.32$$

Interpretation

Lower Gini $\Rightarrow$ higher purity; a pure node has $G = 0$. Gini is the **default** criterion in CART.

The Gini Index

Formula

$$G = \sum_{k=1}^K \hat{p}_{mk}(1 - \hat{p}_{mk}) = 1 - \sum_{k=1}^K \hat{p}_{mk}^2$$

Node: 8 Spam, 2 Not Spam

$$\hat{p}_1 = 0.8, \hat{p}_2 = 0.2$$

Calculation

$$G = 1 - (0.8^2 + 0.2^2) = 1 - 0.68 = 0.32$$

Interpretation

Lower Gini $\Rightarrow$ higher purity; a pure node has $G = 0$. Gini is the **default** criterion in CART.

Formula

$$G = \sum_{k=1}^K \hat{p}_{mk}(1 - \hat{p}_{mk}) = 1 - \sum_{k=1}^K \hat{p}_{mk}^2$$

Node: 8 Spam, 2 Not Spam

$$\hat{p}_1 = 0.8, \hat{p}_2 = 0.2$$

Calculation

$$G = 1 - (0.8^2 + 0.2^2) = 1 - 0.68 = 0.32$$

Interpretation

Lower Gini $\Rightarrow$ higher purity; a pure node has $G = 0$. Gini is the **default** criterion in CART.

Entropy (Cross-Entropy)

Formula

$$H = - \sum_{k=1}^K \hat{p}_{mk} \log_2(\hat{p}_{mk})$$

Entropy is 0 for a perfectly pure node and maximal when all classes are equally likely.

Example

$$\hat{p}_1 = 0.8, \hat{p}_2 = 0.2$$

Calculation

$$H = -(0.8 \log_2 0.8 + 0.2 \log_2 0.2) = 0.722$$

Entropy (Cross-Entropy)

Formula

$$H = - \sum_{k=1}^K \hat{p}_{mk} \log_2(\hat{p}_{mk})$$

Entropy is 0 for a perfectly pure node and maximal when all classes are equally likely.

Example

$$\hat{p}_1 = 0.8, \hat{p}_2 = 0.2$$

Calculation

$$H = -(0.8 \log_2 0.8 + 0.2 \log_2 0.2) = 0.722$$

Comparing the Three Impurity Measures

Measure	Formula	Characteristics
Classification error	$1 - \max_k (\hat{p}_k)$	Simple, but less sensitive to changes in class probabilities.
Gini index	$1 - \sum_k \hat{p}_k^2$	Default in CART; sensitive and computationally efficient.
Entropy	$-\sum_k \hat{p}_k \log_2 \hat{p}_k$	Information-theoretic measure of uncertainty; used by ID3, C4.5.

Why not misclassification rate for growing the tree?

Gini and entropy are *differentiable-in-spirit* and more sensitive to node purity than E : they can favor a split that produces one pure and one impure child even when E is unchanged, which is usually what we want while growing. E is typically preferred only when the final goal is *prediction accuracy* (e.g. pruning by test error).

Comparing the Three Impurity Measures

Measure	Formula	Characteristics
Classification error	$1 - \max_k (\hat{p}_k)$	Simple, but less sensitive to changes in class probabilities.
Gini index	$1 - \sum_k \hat{p}_k^2$	Default in CART; sensitive and computationally efficient.
Entropy	$-\sum_k \hat{p}_k \log_2 \hat{p}_k$	Information-theoretic measure of uncertainty; used by ID3, C4.5.

Why not misclassification rate for growing the tree?

Gini and entropy are *differentiable-in-spirit* and more sensitive to node purity than E : they can favor a split that produces one pure and one impure child even when E is unchanged, which is usually what we want while growing. E is typically preferred only when the final goal is *prediction accuracy* (e.g. pruning by test error).

When Does a Tree Beat a Linear Model?

Linear model wins when

- ▶ The true relationship is close to linear/additive.
- ▶ Interpretation of *coefficients* matters.
- ▶ Signal-to-noise ratio is low — fewer parameters, less variance.

Tree wins when

- ▶ The true decision boundary is highly non-linear or has sharp interactions.
- ▶ Predictors mix numeric and categorical types naturally.
- ▶ Interpretation as a *sequence of rules* is desired.

No free lunch

Neither dominates: an additive true function is fit almost perfectly by a hyperplane but only approximated (in blocky steps) by a tree, and vice versa for a genuinely partitioned truth. Compare on held-out error.

When Does a Tree Beat a Linear Model?

Linear model wins when

- ▶ The true relationship is close to linear/additive.
- ▶ Interpretation of *coefficients* matters.
- ▶ Signal-to-noise ratio is low — fewer parameters, less variance.

Tree wins when

- ▶ The true decision boundary is highly non-linear or has sharp interactions.
- ▶ Predictors mix numeric and categorical types naturally.
- ▶ Interpretation as a *sequence of rules* is desired.

No free lunch

Neither dominates: an additive true function is fit almost perfectly by a hyperplane but only approximated (in blocky steps) by a tree, and vice versa for a genuinely partitioned truth. Compare on held-out error.

Advantages and Disadvantages of Trees

Advantages

- ▶ Easy to explain — arguably *more* interpretable than linear regression.
- ▶ Mirror human decision-making (a flowchart of questions).
- ▶ Handle qualitative predictors without dummy encoding.
- ▶ Displayed graphically, easily understood by non-experts.

Disadvantages

- ▶ Generally *lower predictive accuracy* than other regression/classification approaches.
- ▶ **High variance / instability:** a small change in the training data can produce a very different tree (a different early split cascades down).
- ▶ Non-smooth, blocky decision boundaries.

The instability problem

Because splits are chosen greedily and hierarchically, an early split determines every-

Advantages and Disadvantages of Trees

Advantages

- ▶ Easy to explain — arguably *more* interpretable than linear regression.
- ▶ Mirror human decision-making (a flowchart of questions).
- ▶ Handle qualitative predictors without dummy encoding.
- ▶ Displayed graphically, easily understood by non-experts.

Disadvantages

- ▶ Generally *lower predictive accuracy* than other regression/classification approaches.
- ▶ **High variance / instability**: a small change in the training data can produce a very different tree (a different early split cascades down).
- ▶ Non-smooth, blocky decision boundaries.

The instability problem

Because splits are chosen greedily and hierarchically, an early split determines every-

In Practice: Fitting Trees with scikit-learn

```
from sklearn.tree import (
    DecisionTreeRegressor,
    DecisionTreeClassifier)

# Regression: predict log(Salary)
reg = DecisionTreeRegressor(
    max_depth=3,
    ccp_alpha=0.01)
# cost-complexity alpha
).fit(X_train, y_train)

# Classification: Gini vs entropy
clf = DecisionTreeClassifier(
    criterion="gini", # or "entropy"
    ccp_alpha=0.01)
).fit(X_train, y_train)
```

Mapping to theory

- ▶ `max_depth` / `min_samples_leaf`: crude early-stopping controls.
- ▶ `ccp_alpha`: directly implements cost-complexity pruning,

$$C_\alpha(T) = \text{RSS}(T) + \alpha|T|.$$
- ▶ `criterion`: choose Gini or entropy for splitting a classification tree.

Habit

Use `cost_complexity_pruning_path` (or CV over `ccp_alpha`) to pick α



SECTION 7

Summary

Summary & What Comes Next

Key takeaways

- ▶ Trees recursively partition the predictor space; regression trees minimize RSS, classification trees minimize impurity (Gini / entropy / error rate).
- ▶ Cost-complexity pruning, $C_\alpha(T) = \text{RSS}(T) + \alpha|T|$ with α chosen by CV, controls the bias–variance tradeoff.
- ▶ Trees are highly interpretable but a *single* tree has high variance — small data perturbations can restructure it entirely.

Next session: Bagging, Random Forests, and Boosting

If one tree is unstable, **average many of them**. Bagging and random forests fit many trees on resampled/derandomized data and average their predictions; boosting grows trees sequentially to correct earlier errors. Both convert a single tree's high variance into a much more accurate ensemble.

Key takeaways

- ▶ Trees recursively partition the predictor space; regression trees minimize RSS, classification trees minimize impurity (Gini / entropy / error rate).
- ▶ Cost-complexity pruning, $C_\alpha(T) = \text{RSS}(T) + \alpha|T|$ with α chosen by CV, controls the bias–variance tradeoff.
- ▶ Trees are highly interpretable but a *single* tree has high variance — small data perturbations can restructure it entirely.

Next session: Bagging, Random Forests, and Boosting

If one tree is unstable, **average many of them**. Bagging and random forests fit many trees on resampled/derandomized data and average their predictions; boosting grows trees sequentially to correct earlier errors. Both convert a single tree's high variance into a much more accurate ensemble.

Tree-Based Methods

From one greedy tree to an ensemble of trees

Questions & discussion welcome

